Group A: Design and Analysis of Algorithms (410241)

Unit-wise viva questions with short spoken-style answers, then questions on practicals 1–5.

Unit I: Algorithms and Problem Solving

Must know: definition and properties of an algorithm, why correctness must be proved, loop invariants, problem-solving strategies, the common complexity classes, Towers of Hanoi.

Q1. What is an algorithm? A finite sequence of unambiguous instructions that solves a problem: it takes zero or more inputs and produces at least one output in a finite amount of time.

Q2. What are the properties (characteristics) of an algorithm? Input, output, definiteness (each step is clear), finiteness (it terminates), effectiveness (each step is basic enough to be carried out), and generality.

Q3. Algorithm vs program? An algorithm is a language-independent design written in steps or pseudocode and is analysed for efficiency. A program is its implementation in a specific language that runs on a machine.

Q4. Why are algorithms called a "technology"? Because a better algorithm can give more speed-up than faster hardware. Sorting 10 million numbers with merge sort (n log n) on a slow PC beats insertion sort (n^2) on a computer 100× faster.

Q5. What are the steps in designing an algorithm? Understand the problem, pick a model and data structures, choose a design strategy (divide and conquer, greedy, DP, backtracking, and so on), write the algorithm, prove it correct, analyse its time and space, then code and test it.

Q6. Why must an algorithm's correctness be proved? Isn't testing enough? Testing only shows the algorithm works for the inputs you tried; it cannot show the absence of errors. A proof (by induction or a loop invariant) shows it works for every valid input.

Q7. What is a loop invariant? A condition that is true before the loop starts (initialisation), stays true after every iteration (maintenance), and together with the exit condition proves the result (termination). For insertion sort: "A[0..i-1] is always sorted."

Q8. What are iterative algorithm design issues? Choosing correct loop bounds and initial values, guaranteeing termination, keeping the loop invariant, avoiding off-by-one errors, and minimising the work done inside the loop.

Q9. How are problems classified? By type: decision (yes/no), optimisation (best value), search, counting and enumeration. By difficulty: tractable (polynomial time) and intractable (exponential or worse), or undecidable (no algorithm exists, e.g. the halting problem).

Q10. Name common problem-solving strategies. Brute force, divide and conquer, decrease and conquer, transform and conquer, greedy, dynamic programming, backtracking, branch and bound, randomized algorithms.

Q11. List the time complexity classes from fastest to slowest. $O(1)$ constant < $O(\log n)$ logarithmic (binary search) < $O(n)$ linear (linear search) < $O(n \log n)$ (merge sort) < $O(n^2)$ quadratic (bubble sort) < $O(n^3)$ cubic (matrix multiplication) < $O(2^n)$ exponential (subsets, recursive Fibonacci) < $O(n!)$ factorial (permutations, brute-force TSP).

Q12. Explain Towers of Hanoi. Move n disks from peg A to peg C using peg B, moving one disk at a time and never placing a larger disk on a smaller one. Recursively: move $n-1$ disks A→B, move the largest disk A→C, move $n-1$ disks B→C.

Q13. What is the recurrence and complexity of Towers of Hanoi? $T(n) = 2T(n-1) + 1$ with $T(1) = 1$, so $T(n) = 2^n - 1$ moves, which is $O(2^n)$. Space is $O(n)$ for the recursion stack. For example, 3 disks need 7 moves and 64 disks need $2^{64} - 1$ moves.

Q14. Recursion vs iteration? Recursion is a function calling itself with a base case; it is elegant but uses stack space and has call overhead. Iteration uses loops and constant extra space. Every recursive algorithm can be written iteratively using an explicit stack.

Unit II: Analysis of Algorithms and Complexity Theory

Must know: best/worst/average case, the five asymptotic notations, counting the dominant operation, P, NP, NP-complete, NP-hard, reductions, vertex cover, 3-SAT, Hamiltonian cycle.

Notation	Meaning	Read as
$O(g(n))$	$f(n) \leqslant c \cdot g(n)$ for all $n \geqslant n_0$	upper bound, “at most”
$\Omega(g(n))$	$f(n) \geqslant c \cdot g(n)$ for all $n \geqslant n_0$	lower bound, “at least”

Notation	Meaning	Read as
$\Theta(g(n))$	$c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n)$	tight bound, “exactly of order”
$o(g(n))$	f grows strictly slower than g	strict upper bound
$\omega(g(n))$	f grows strictly faster than g	strict lower bound

Q1. What is input size? The quantity that determines how much work the algorithm does: the number of elements for sorting, the number of vertices and edges for a graph, or the number of bits for arithmetic on large numbers.

Q2. Best, worst and average case? The minimum, maximum and expected running time over all inputs of size n. For linear search: best O(1) (first element), worst O(n) (last or absent), average about n/2 comparisons, i.e. O(n).

Q3. What is the dominant (basic) operation? The operation executed most often, which therefore determines the growth rate, such as the comparison in sorting or the multiplication in matrix multiplication. Analysis counts it and ignores lower-order work.

Q4. Why do we use asymptotic notation? It describes growth for large n independent of machine, language and constants, so algorithms can be compared fairly. $3n^2 + 5n + 10$ is simply $O(n^2)$.

Q5. How do you analyse a recursive algorithm? Write a recurrence relation and solve it by substitution, the recursion tree method or the Master theorem. Merge sort: $T(n) = 2T(n/2) + n = O(n \log n)$.

Q6. State the Master theorem. For $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{(\log_b a)}$: if f is smaller, $T = \Theta(n^{(\log_b a)})$; if they are equal, $T = \Theta(n^{(\log_b a)} \cdot \log n)$; if f is larger (with a regularity condition), $T = \Theta(f(n))$.

Q7. Polynomial vs non-polynomial problems? Polynomial problems have algorithms running in $O(n^k)$ for some constant k (sorting, shortest path), and are called tractable. Non-polynomial ones have only exponential algorithms known (TSP, subset sum), and are called intractable.

Q8. Deterministic vs non-deterministic algorithm? A deterministic algorithm has exactly one next step at each point. A non-deterministic algorithm can “guess” the right choice (in theory, exploring all choices in parallel), then verifies the guess in polynomial time.

Q9. What is the class P? Decision problems solvable by a deterministic algorithm in polynomial time, e.g. sorting, searching, minimum spanning tree, shortest path.

Q10. What is the class NP? Decision problems whose "yes" answer can be verified in polynomial time given a certificate, or equivalently solved by a non-deterministic algorithm in polynomial time. Every P problem is in NP.

Q11. Is P = NP? Unknown. It's the most famous open problem in computer science (a Millennium Prize problem). Most researchers believe P ≠ NP.

Q12. What is polynomial reduction? Transforming every instance of problem A into an instance of problem B in polynomial time so that the answers match (A ⩽p B). If B is easy, A is easy; if A is hard, B is hard.

Q13. What is NP-complete? A problem that is in NP and to which every NP problem reduces in polynomial time. Examples: SAT (proved by Cook's theorem), 3-SAT, vertex cover, clique, Hamiltonian cycle, subset sum.

Q14. What is NP-hard? A problem at least as hard as every NP problem, but not necessarily in NP (its solution may not be verifiable in polynomial time, or it may not be a decision problem). Examples: optimisation TSP, the halting problem.

Q15. Explain the 3-SAT problem. Given a Boolean formula in CNF where every clause has exactly three literals, e.g. (x1 ∨ ¬x2 ∨ x3) ∧ (¬x1 ∨ x2 ∨ x4), is there an assignment that makes it true? It's NP-complete and is the usual starting point for reductions.

Q16. Explain vertex cover. A set of vertices such that every edge touches at least one of them. The decision problem "Is there a cover of size ⩽ k?" is NP-complete (3-SAT reduces to it). A simple 2-approximation picks both ends of any uncovered edge.

Q17. Explain the Hamiltonian cycle problem. Is there a cycle that visits every vertex exactly once and returns to the start? It's NP-complete (the syllabus lists it under NP-hard; its optimisation cousin, TSP, is NP-hard). Contrast an Euler circuit (every edge exactly once), which is solvable in polynomial time.

Q18. Draw the P, NP, NP-complete, NP-hard diagram. P sits inside NP. NP-complete is the intersection of NP and NP-hard. NP-hard extends beyond NP. If any NP-complete problem is solved in polynomial time, then P = NP.

Unit III: Greedy and Dynamic Programming

Must know: greedy principle and control abstraction, fractional knapsack, job sequencing with deadlines, activity selection; DP principle (optimal substructure + overlapping subproblems), binomial coefficients, OBST, 0/1 knapsack, matrix chain multiplication; case study: MST ("rail tracks connecting all cities").

	Greedy	Dynamic Programming
Choice	One locally best choice per step, never reconsidered	Considers all choices, keeps the best via stored results
Optimal?	Only when the greedy-choice property holds	Always optimal if optimal substructure holds
Speed / memory	Faster, little memory	Slower, needs a table
Examples	Fractional knapsack, Huffman, Prim, Kruskal, Dijkstra, job sequencing	0/1 knapsack, OBST, matrix chain, LCS, Floyd–Warshall, Bellman–Ford

Q1. What is the greedy method? Build the solution step by step, always picking the option that looks best right now (locally optimal), hoping it leads to a globally optimal solution. Choices are never undone.

Q2. Write the control abstraction of greedy. `Greedy(a, n): solution = ∅; for i = 1 to n: x = Select(a); if Feasible(solution ∪ x): solution = solution ∪ x; return solution`. Its time is dominated by Select, usually a sort, so O(n log n).

Q3. What two properties make greedy correct? The greedy-choice property (a locally optimal choice is part of some optimal solution) and optimal substructure (an optimal solution contains optimal solutions to subproblems).

Q4. Fractional knapsack: how and what complexity? Sort items by value/weight ratio in descending order; take whole items while they fit, then a fraction of the next item. O(n log n). Greedy is optimal here because items are divisible.

Q5. Explain job sequencing with deadlines. Each job takes one unit of time and has a deadline and a profit. Sort jobs by profit descending, then place each job in the latest free slot on or before its deadline; skip it if no slot is free. $O(n^2)$ simple, O(n log n) with disjoint sets.

Q6. Explain activity selection. Choose the maximum number of non-overlapping activities. Sort by finish time, pick the first, then repeatedly pick the next activity whose start ≥ the last finish. O(n log n).

Q7. "Rail tracks connecting all cities": which algorithm? Minimum spanning tree, connecting all cities with minimum total track length. Prim's grows one tree from a vertex (O(E log V) with a heap); Kruskal's adds the cheapest edges that don't form a cycle using union-find (O(E log E)). Both are greedy.

Q8. What is dynamic programming? Solve a problem by breaking it into overlapping subproblems, solving each once, and storing the answers in a table (memoization top-down, or tabulation bottom-up) so they are reused instead of recomputed.

Q9. What two properties must a problem have for DP? Optimal substructure and overlapping subproblems. Without overlap (as in merge sort), plain divide and conquer is enough.

Q10. Memoization vs tabulation? Memoization is top-down recursion that caches results; it computes only the needed subproblems. Tabulation is bottom-up iteration that fills the table from the smallest subproblems; it has no recursion overhead.

Q11. Control abstraction of DP? Characterise the structure of an optimal solution, define the value recursively (the recurrence), compute values bottom-up in a table, then construct the solution by tracing back through the table.

Q12. Binomial coefficient using DP? $C(n,k) = C(n-1,k-1) + C(n-1,k)$, with $C(n,0) = C(n,n) = 1$. Fill Pascal's triangle row by row: $O(n \cdot k)$ time, compared with exponential time for plain recursion.

Q13. 0/1 knapsack recurrence and complexity? $V[i][w] = V[i-1][w]$ if $w_i > w$, else $\max(V[i-1][w], v_i + V[i-1][w-w_i])$. $O(n \cdot W)$ time and space; this is pseudo-polynomial because it depends on the value of W, not its number of bits.

Q14. What is an Optimal Binary Search Tree (OBST)? A BST built from keys with known search probabilities so that the expected search cost is minimum. DP: cost[i][j] = min over roots r of (cost[i][r−1] + cost[r+1][j]) + sum of probabilities from i to j. $O(n^3)$, or $O(n^2)$ with Knuth's optimisation.

Q15. Matrix chain multiplication? Find the parenthesisation of $A1 \cdot A2 \cdot \ldots \cdot An$ that minimises scalar multiplications (the result is the same, the cost isn't). m[i][j] = min over k of (m[i][k] + m[k+1][j] + $p_{i-1} \cdot p_k \cdot p_j$). $O(n^3)$ time, $O(n^2)$ space.

Q16. Why is DP better than recursion for Fibonacci? Plain recursion recomputes the same F(k) values exponentially often ($O(2^n)$). DP stores each F(k) once, giving O(n) time; keeping only the last two values gives O(1) space.

Unit IV: Backtracking and Branch and Bound

Must know: state-space tree, the backtracking control abstraction, 8-queens, graph colouring, sum of subsets; B&B with FIFO, LIFO and LC (least-cost) search, TSP, knapsack; case study: airline crew scheduling.

	Backtracking	Branch and Bound
Search order	Depth-first	BFS (FIFO), DFS (LIFO) or best-first (LC)
Prunes using	Feasibility (bounding/constraint function)	Bound on the best possible value of a node
Typical problems	Decision / all solutions: N-queens, colouring, subset sum	Optimisation: TSP, 0/1 knapsack, job assignment
Memory	Low (one path on the stack)	Higher (a queue of live nodes)

Q1. What is backtracking? A systematic DFS of the state-space tree that builds a solution one component at a time and abandons (backtracks from) a partial solution as soon as it violates a constraint.

Q2. What is a state-space tree? A tree in which each node is a partial solution and each edge is a choice. The root is the empty solution, and leaves are complete candidates. Live nodes are still being expanded; dead nodes have been pruned or fully explored.

Q3. Control abstraction of backtracking? `Backtrack(k): for each choice x[k] in T(x[1..k−1]): if Bounded(x[1..k]): if x[1..k] is a solution: print it; else Backtrack(k+1)`. Worst case it is exponential, but pruning makes it practical.

Q4. Explain the 8-queens problem. Place 8 queens on an 8×8 board so that no two share a row, column or diagonal. Place one queen per row and try each column; if no column is safe, backtrack to the previous row. There are 92 solutions (12 distinct up to symmetry).

Q5. How do you check that two queens are on the same diagonal? $|r1 - r2| = |c1 - c2|$. Same column: $c1 = c2$. One queen per row removes the row check.

Q6. Complexity of N-queens? $O(N!)$ worst case, because row k has at most $N - k + 1$ safe columns, and checking safety costs $O(N)$. Solutions exist for every N except 2 and 3.

Q7. Explain graph colouring. Colour the vertices with at most m colours so that adjacent vertices differ. Backtracking assigns vertex k the first colour not used by its neighbours, and backtracks if none is available. The smallest such m is the chromatic number. Uses: map colouring, register allocation, exam timetables. $O(m^n)$ worst case.

Q8. Explain sum of subsets. Find subsets of positive numbers that add up to M. Sort the numbers; at each level include or exclude the next number. Prune when the current sum plus the next item > M, or the current sum plus all remaining < M. Example: {5, 10, 12, 13, 15,

18}, M = 30 gives {5,10,15}, {5,12,13}, {12,18}.

Q9. What is branch and bound? A state-space search for optimisation problems. Each node gets a bound (the best value achievable in its subtree), and a node is killed when its bound can't beat the best solution found so far.

Q10. FIFO vs LIFO vs LC branch and bound? FIFO keeps live nodes in a queue (BFS). LIFO uses a stack (DFS-like). LC (least cost) uses a priority queue ordered by a cost estimate $\hat{c}(x)$, so it always expands the most promising node first and usually finds the optimum fastest.

Q11. What is the cost function ĉ(x) in LC search? $\hat{c}(x) = f(h(x)) + \hat{g}(x)$: the cost already incurred to reach x plus an estimate of the remaining cost to an answer node. It plays the same role as the heuristic in A*.

Q12. TSP using branch and bound? Use a reduced cost matrix: subtract each row's minimum, then each column's minimum; the sum subtracted is a lower bound. Each branch fixes the next city, re-reduces the matrix and adds to the bound. Expand the node with the least bound until a full tour is reached. Worst case is still O(n!), but pruning cuts most branches.

Q13. 0/1 knapsack using branch and bound? The upper bound of a node = current profit + the fractional (greedy) knapsack value of the remaining capacity. Branch on include/exclude for the next item, and prune nodes whose bound ⩽ the best profit found so far.

Q14. Airline crew scheduling: which strategy and why? Assigning crews to flights while minimising cost under rules (rest hours, base cities, legal duty limits) is a combinatorial optimisation problem. It's modelled as set covering and solved with branch and bound (branch-and-price in industry), because the bounds prune the enormous number of possible pairings.

Q15. Backtracking vs brute force? Brute force generates every complete candidate and then checks it. Backtracking checks constraints on partial solutions and prunes whole subtrees early, so it explores far fewer nodes.

Unit V: Amortized Analysis and Other Algorithms

Must know: aggregate, accounting and potential methods; binary counter and stack examples; time–space trade-off; tractable vs intractable; randomized and approximation algorithms; embedded scheduling and sorting; case study: cutting stock.

Q1. What is amortized analysis? The average cost of each operation over a worst-case sequence of n operations. It gives a tighter bound than multiplying the worst single-operation cost by n. It is not average-case analysis: no probability is involved.

Q2. Explain the aggregate method. Find the total cost T(n) of n operations in the worst case; the amortized cost per operation is T(n)/n. Example: n increments of a binary counter flip at most 2n bits in total, so each increment is O(1) amortized.

Q3. Explain the accounting (banker's) method. Charge each operation an amortized cost that may be more than its actual cost; the extra is stored as credit on data elements and pays for expensive operations later. The credit must never go negative. Binary counter: charge 2 for setting a bit to 1 (1 pays now, 1 is saved to reset it later).

Q4. Explain the potential method. Define a potential function Φ on the data structure (stored energy), with $\Phi(D_0) = 0$ and $\Phi \geq 0$. Amortized cost = actual cost + Φ(after) − Φ(before). For a binary counter, Φ = number of 1-bits.

Q5. Amortized cost of a stack with multipop? PUSH and POP are O(1); MULTIPOP(k) is O(min(k, size)). Each element can be popped at most once after being pushed, so n operations cost O(n) in total, which is O(1) amortized per operation.

Q6. Other examples of amortized O(1)? Appending to a dynamic array that doubles when full (Python list, Java ArrayList), and union-find with path compression (nearly constant).

Q7. What is a time–space trade-off? Using more memory to save time or vice versa. Examples: DP tables and memoization (more space, less time), lookup tables, hashing, and compressed data (less space, more time to decompress).

Q8. Tractable vs intractable problems? Tractable problems have polynomial-time algorithms (sorting, shortest path). Intractable problems need super-polynomial time as far as we know (TSP, SAT) and become impractical as n grows.

Q9. What is a randomized algorithm? An algorithm that makes random choices during execution. Las Vegas algorithms are always correct but their running time is random (randomized quicksort, expected O(n log n)). Monte Carlo algorithms have bounded time but may be wrong with small probability (Miller–Rabin primality test).

Q10. Why randomize quicksort? Choosing a random pivot makes the $O(n^2)$ worst case extremely unlikely for any input, including already-sorted data. Expected time is O(n log n).

Q11. What is an approximation algorithm? A polynomial-time algorithm for an NP-hard optimisation problem that guarantees a solution within a factor of the optimum (the approximation ratio ρ). Examples: vertex cover 2-approximation, metric TSP via MST

(factor 2), greedy set cover (ln n factor).

Q12. What is an embedded algorithm, and what constraints matter? An algorithm that runs on an embedded device such as a microcontroller in a car, phone or IoT sensor. It must respect limited memory, limited CPU, battery power and real-time deadlines, so it prefers in-place, predictable and low-power solutions.

Q13. Power-optimised scheduling in embedded systems? Dynamic voltage and frequency scaling (DVFS): run tasks at the lowest speed and voltage that still meets their deadlines, because power scales roughly with $V^2 \cdot f$. Combined with real-time schedulers such as EDF (Earliest Deadline First) or RMS (Rate Monotonic Scheduling), and with idle-time shutdown.

Q14. Which sorting algorithm suits embedded systems? An in-place algorithm with predictable time and little memory: insertion sort for small or nearly sorted data, heap sort for a guaranteed O(n log n) with O(1) extra space, shell sort as a compromise. Merge sort needs O(n) extra memory, and quicksort has an $O(n^2)$ worst case and recursion stack.

Q15. What is the cutting stock problem? Cut standard-size rolls or sheets into ordered smaller pieces while minimising waste or the number of rolls used. It's NP-hard (related to bin packing). Solved with greedy heuristics (first-fit decreasing), dynamic programming for the knapsack subproblem, and linear programming (column generation).

Unit VI: Multithreaded and Distributed Algorithms, String Matching

Must know: work, span, speed-up and parallelism; spawn, sync and parallel for; race conditions; multithreaded matrix multiplication and merge sort; distributed BFS and MST; naive and Rabin–Karp string matching; case study: plagiarism detection.

Q1. What is a multithreaded algorithm? An algorithm that runs on a multi-core shared-memory machine by splitting work into threads. In pseudocode this uses `spawn` (start a parallel child), `sync` (wait for children) and `parallel for`.

Q2. Define work, span, speed-up and parallelism. Work T_1 is the total time on one processor. Span T_∞ is the time on infinitely many processors (the critical path). Speed-up is T_1/T_p on P processors. Parallelism is T_1/T_∞, the maximum possible speed-up.

Q3. What are the work law and span law? Work law: $T_p \geq T_1/P$. Span law: $T_p \geq T_\infty$. A greedy scheduler achieves $T_p \leq T_1/P + T_\infty$.

Q4. What is a race condition? When two threads access the same memory location concurrently, at least one writes, and the result depends on the timing. Example: two threads doing x = x + 1 can lose an update. Fix it with locks or mutexes, atomic operations, or by designing threads to work on independent data (determinacy).

Q5. How do parallel loops work? `parallel for i = 1 to n` lets all iterations run concurrently. It is implemented by recursively splitting the range with spawn, which adds O(log n) to the span. Iterations must be independent.

Q6. Multithreaded matrix multiplication? The two outer loops (rows i, columns j) become parallel for; each C[i][j] is independent. Work $\Theta(n^3)$, span $\Theta(n)$, parallelism $\Theta(n^2)$. The divide-and-conquer version (spawn the 8 sub-multiplications) has span $\Theta(\log^2 n)$.

Q7. Multithreaded merge sort? Spawn the two recursive sorts and sync before merging. With a serial merge, work is $\Theta(n \log n)$ and span is $\Theta(n)$, so parallelism is only $\Theta(\log n)$. A parallel merge (binary search split) reduces the span to $\Theta(\log^3 n)$.

Q8. What is a distributed algorithm? An algorithm that runs on several independent computers (nodes) that don't share memory and communicate only by messages over a network. Concerns include message complexity, rounds, failures and the lack of a global clock.

Q9. Distributed BFS? The root sends an "explore" message to its neighbours; each node, on first receiving it, records the sender as its parent and its level, then forwards the message to its other neighbours. The synchronous version takes O(D) rounds (D = diameter) and O(E) messages.

Q10. Distributed MST? The GHS (Gallager–Humblet–Spira) algorithm: every node starts as its own fragment; each fragment finds its minimum-weight outgoing edge and merges along it (like Borůvka's algorithm), until one fragment remains. O(N log N + E) messages.

Q11. What is string matching? Finding all positions where a pattern P (length m) occurs in a text T (length n). Uses: search in editors, DNA sequence matching, spam filtering, plagiarism detection.

Q12. Naive string matching: how and what complexity? Slide the pattern over every shift s from 0 to n−m and compare character by character. $O((n-m+1)\cdot m)$, which is O(nm) in the worst case (e.g. T = aaaa…a, P = aa…ab). No preprocessing.

Q13. How does Rabin-Karp work? Compare hash values instead of strings. Compute the hash of the pattern and of each m-length window of the text using a rolling hash (remove the leading character, add the next one in O(1)), modulo a prime q. Only when the hashes match are the characters compared.

Q14. Complexity of Rabin–Karp, and what is a spurious hit? Average O(n + m); worst case O(nm) when many hashes collide. A spurious hit is a hash match where the strings actually differ, which is why a character check follows. A large prime q keeps spurious hits rare.

Q15. How is string matching used in plagiarism detection? Break documents into overlapping k-grams (fingerprints), hash them with Rabin–Karp-style rolling hashes, and count the matching hashes between documents. Many shared fingerprints suggest copying. Rabin–Karp suits this because it can search for many patterns at once.

Lab practicals 1–5: questions on your code

#	Practical	Strategy	Time	Space
1	Fibonacci, iterative	Iteration	O(n)	O(1)
1	Fibonacci, recursive	Recursion	$O(2^n)$	O(n) stack
2	Huffman encoding	Greedy + min-heap	O(n log n)	O(n)
3	Fractional knapsack	Greedy by value/weight	O(n log n)	O(n)
4	0/1 knapsack	Dynamic programming	O(n·W)	O(n·W)
5	N-queens, first queen fixed	Backtracking	O(N!)	$O(N^2)$ board

Practical 1: Fibonacci

- **Why is the recursive version so slow?** It recomputes the same subproblems; F(n) makes about 2^n calls (177 calls for n = 10, 242,785 for n = 25).
- **Space of the recursive version?** O(n), the maximum depth of the call stack.
- **How would you make recursion fast?** Memoization (store computed values, e.g. `functools.lru_cache`), which gives O(n).
- **Fastest known method?** Matrix exponentiation $[[1,1],[1,0]]^n$ or fast doubling, O(log n).
- **What happens for a very large n in recursion?** It becomes far too slow and Python hits its recursion limit (about 1000). That's why the program skips recursion for n > 35.
- **Closed form?** Binet's formula: $F(n) = (\varphi^n - \psi^n)/\sqrt{5}$, where $\varphi = 1.618\ldots$ (the golden ratio).

Practical 2: Huffman encoding

- **Why is Huffman coding greedy?** At each step it merges the two lowest-frequency nodes, the locally best choice.
- **What is a prefix code, and why does it matter?** No code is a prefix of another, so the bit string decodes uniquely without separators.
- **Why a min-heap?** It gives the two smallest frequencies in O(log n), so the n−1 merges take O(n log n) in total.
- **Is Huffman lossless or lossy?** Lossless; decoding gives back the exact original (the program decodes to prove it).
- **Is the Huffman tree unique?** No. Ties can be broken differently, giving different codes, but the total encoded length is the same and optimal.
- **Where is it used?** ZIP/DEFLATE, JPEG, MP3, PNG.
- **Fixed-length vs variable-length coding?** Fixed uses $\lceil \log_2 n \rceil$ bits per symbol; Huffman uses short codes for frequent symbols, so the average is smaller.

Practical 3: Fractional knapsack

- **What is the greedy criterion?** The highest value/weight ratio first. Picking by highest value or lowest weight alone isn't always optimal.
- **Why is greedy optimal here but not for 0/1?** Items can be split, so the remaining capacity is always filled with the best possible ratio.
- **Where does the O(n log n) come from?** Sorting by ratio; the selection loop is O(n).
- **Real-life example?** Loading divisible goods (grain, liquids, gold dust) into limited capacity.

Practical 4: 0/1 knapsack (DP)

- **Explain your DP table.** Rows are items 0..n and columns are capacities 0..W; cell [i][w] is the best value using the first i items with capacity w.
- **How do you find which items were selected?** Trace back from [n][W]: if V[i][w] ≠ V[i−1][w], item i was taken, so subtract its weight and move up a row.
- **Why is O(nW) called pseudo-polynomial?** W is a number, not an input size; doubling the number of bits in W squares the work.
- **Can you reduce space?** Yes, to O(W) with a single 1-D array filled from right to left (but then you can't trace back the items).

- **Show greedy failing.** Items (60,10), (100,20), (120,30), W = 50: greedy by ratio gives 160, DP gives 220.
- **Other approaches?** Brute force over 2^n subsets, branch and bound with a fractional upper bound, backtracking.

Practical 5: N-queens with the first queen fixed

- **What does `is_safe` check, and why not rows?** The column above, the upper-left diagonal and the upper-right diagonal. Each row gets exactly one queen, so rows never clash, and rows below are still empty.
- **Where exactly does backtracking happen?** `board[row][col] = 0` after the recursive call fails: the queen is removed and the next column is tried.
- **Why does N = 4 with the first queen in column 1 fail?** No arrangement exists with a queen in a corner of a 4×4 board; the only solutions are 2-4-1-3 and 3-1-4-2.
- **How many solutions does 8-queens have?** 92 (12 fundamental).
- **How could you make the safety check O(1)?** Keep boolean arrays for columns, diagonals (r − c) and anti-diagonals (r + c).
- **Which N have no solution?** 2 and 3.

Mini Project: Multithreaded Matrix Multiplication

Your result: for small n, sequential is fastest; thread per cell is slowest (n^2 threads); Python thread per row gives no speed-up because of the GIL; multiprocessing reached 1.5× at n = 200 on 2 cores.

- **Complexity of normal matrix multiplication?** $\Theta(n^3)$: n^2 cells × n multiplications each. Strassen's algorithm is O(n^2.81).
- **Thread per row vs thread per cell?** Row: n threads, each doing $\Theta(n^2)$ work. Cell: n^2 threads, each doing $\Theta(n)$ work. Cell has maximum parallelism but huge thread-creation overhead, so it's slower in practice.
- **Why didn't Python threads speed it up?** The GIL (Global Interpreter Lock) allows only one thread to execute Python bytecode at a time, so CPU-bound threads run concurrently, not in parallel. They help only for I/O-bound work.
- **How did you get real parallelism?** `multiprocessing.Pool`: separate processes, each with its own interpreter and GIL, one block of rows per CPU core.
- **Why is multiprocessing slower for small n?** Process start-up and copying (pickling) the matrices cost more than the computation.

- **Is there a race condition in your code?** No. Each thread writes to a different row or cell of C, and A and B are only read. With a shared `+=` on one cell, you'd need a lock.
- **Concurrency vs parallelism?** Concurrency interleaves tasks and makes progress on several at once; parallelism runs them at the same instant on different cores.
- **Thread vs process?** Threads share memory and are cheap to create. Processes have separate memory, are safer and can be truly parallel in Python, but cost more to start and communicate.
- **What do `start()` and `join()` correspond to?** `spawn` and `sync` in the CLRS multithreaded pseudocode.
- **Work, span and parallelism for parallel matrix multiplication?** Work $\Theta(n^3)$, span $\Theta(n)$ with parallel i and j loops, parallelism $\Theta(n^2)$.
- **What is Amdahl's law?** Speed-up $\leqslant 1 / (s + (1-s)/P)$, where s is the serial fraction. Thread creation and joining are serial overhead, which caps the speed-up.
- **How many threads should you use?** About the number of CPU cores for CPU-bound work; more just adds context switching.
- **Why the `if __name__ == "__main__":` guard?** On Windows and macOS, child processes re-import the script; without the guard they would spawn processes endlessly.